

## Proyecto 1

### Bases de datos avanzadas

#### Integrantes

Cesar Montoya

Juan Pablo Corena

#### Docente

Edwin Montoya Múnera

Escuela Ciencias Aplicadas e Ingeniería

Trayectoria de Ingeniería de Datos

del programa de Ingeniería de Sistemas

## Objetivo del Trabajo.

El objetivo del trabajo fue analizar, medir y optimizar el rendimiento de consultas SQL sobre un entorno de Big Data, aplicando técnicas de optimización como uso de EXPLAIN y EXPLAIN ANALYZE, creación de índices, particionamiento por rango, reescritura de queries, análisis de concurrencia y performance tuning del servidor PostgreSQL.

Se buscó evaluar el impacto incremental de cada técnica sobre el tiempo de ejecución, uso de buffers y plan de ejecución, con el fin de demostrar cómo una correcta estrategia de optimización mejora la eficiencia, escalabilidad y comportamiento del motor bajo carga y grandes volúmenes de datos.

## Casos de optimización

### 1. Caso de optimización: búsqueda con LIKE '%...%'

Consulta: Buscar clientes cuyo email contenga "customer123"

```
-- Q1 consulta con LIKE
EXPLAIN (ANALYZE, BUFFERS)
SELECT name, email
FROM customer
WHERE email LIKE '%customer123%';
```

```
"Gather (cost=1000.00..18411.33 rows=100 width=41) (actual time=0.187..81.168 rows=1111.00 loops=1)"
" Workers Planned: 2"
" Workers Launched: 2"
" Buffers: shared hit=12193"
" -> Parallel Seq Scan on customer (cost=0.00..17401.33 rows=42 width=41) (actual time=3.907..71.537 rows=370.33 loops=3)"
" Filter: (email ~~ '%customer123% '::text)"
" Rows Removed by Filter: 332963"
" Buffers: shared hit=12193"
"Planning Time: 0.064 ms"
"Execution Time: 81.741 ms"
```

La consulta no podía usar índices B-Tree debido al wildcard inicial (%), generando un Parallel Seq Scan.

Esto provocaba:

- Escaneo completo de la tabla
- Alto consumo de buffers (12193)
- Tiempo de ejecución de 81 ms

La Técnica aplicada fue utilizar la extensión pg\_trgm y un índice GIN:

```
CREATE EXTENSION pg_trgm;
CREATE INDEX idx_cust_email_trgm
ON customer USING gin (email gin_trgm_ops);
```

Y es aqui donde se ve claramente la diferencia:

```
"Bitmap Heap Scan on customer (cost=172.42..546.50 rows=100 width=41) (actual time=26.762..27.607 rows=1111.00 loops=1)"
"  Recheck Cond: (email ~* '%customer123%':text)"
"  Rows Removed by Index Recheck: 20"
"  Heap Blocks: exact=29"
"  Buffers: shared hit=1667"
" -> Bitmap Index Scan on idx_cust_email_trgm (cost=0.00..172.39 rows=100 width=0) (actual time=26.737..26.738 rows=1131.00 loops=1)"
"    Index Cond: (email ~* '%customer123%':text)"
"    Index Searches: 1"
"    Buffers: shared hit=1638"
"Planning:"
"  Buffers: shared hit=1"
"Planning Time: 0.090 ms"
"Execution Time: 28.195 ms"
```

Resultados obtenidos:

| Métrica          | Antes             | Después          |
|------------------|-------------------|------------------|
| Tipo de Scan     | Parallel Seq Scan | Bitmap Heap Scan |
| Buffers          | 12193             | 1667             |
| Heap Blocks      | miles             | 29               |
| Tiempo ejecución | 81 ms             | 28 ms            |

El uso de un índice GIN con pg\_trgm permitió transformar un escaneo completo en un acceso dirigido mediante índice, reduciendo significativamente el uso de memoria y el tiempo de ejecución.

2. Caso de optimización: El Problema de la "Sargabilidad" (Funciones en Columnas)

Consulta: Buscar órdenes realizadas en un año específico (ej. 2023).

```
-- Q2 El Problema de la "Sargabilidad"

-- Consulta problemática (no sargable)
EXPLAIN (ANALYZE, BUFFERS)
SELECT COUNT(*)
FROM orders
WHERE EXTRACT(YEAR FROM order_date) = 2023;
```

```
"Finalize Aggregate (cost=73943.16..73943.17 rows=1 width=8) (actual time=1641.805..1644.150 rows=1.00 loops=1)"
"  Buffers: shared read=41667"
" -> Gather (cost=73942.94..73943.15 rows=2 width=8) (actual time=1641.793..1644.140 rows=3.00 loops=1)"
"    Workers Planned: 2"
"    Workers Launched: 2"
"    Buffers: shared read=41667"
" -> Partial Aggregate (cost=72942.94..72942.95 rows=1 width=8) (actual time=1634.232..1634.234 rows=1.00 loops=3)"
"    Buffers: shared read=41667"
" -> Parallel Seq Scan on orders (cost=0.00..72916.90 rows=10417 width=0) (actual time=0.229..1203.172 rows=332900.67 loops=3)"
"    Filter: (EXTRACT(year FROM order_date) = '2023'::numeric)"
"    Rows Removed by Filter: 1333766"
"    Buffers: shared read=41667"
```

```
"Planning:"
" Buffers: shared hit=26 read=2"
"Planning Time: 0.144 ms"
"Execution Time: 1644.190 ms"
```

Una condición es sargable (Search ARGument ABLE) cuando permite que el optimizador use un índice directamente, si aplicamos una función sobre la columna:

EXTRACT(YEAR FROM order\_date)

PostgreSQL no puede usar un índice normal sobre order\_date porque el índice guarda valores ordenados de:

2021-03-01

2021-03-02

2021-03-03

Pero estamos transformando cada fila en:

2021

2021

2021

Entonces el motor debe: leer cada fila, aplicar la función y comparar el resultado, eso normalmente produce un Seq Scan on orders y es lo que pudimos observar en el query plan.

Para este problema se propone una reescritura de la consulta de la siguiente manera:

```
-- Versión optimizada (sargable)
EXPLAIN (ANALYZE, BUFFERS)
SELECT COUNT(*)
FROM orders
WHERE order_date >= '2023-01-01'
      AND order_date < '2024-01-01';
|
-- índice propuesto
CREATE INDEX idx_orders_order_date
ON orders (order_date);
```

La reescritura transforma la condición en una búsqueda por rango, permitiendo el uso eficiente de un índice B-Tree sobre order\_date, y también se implemento el uso del índice, obteniendo como resultado:

```
"Finalize Aggregate (cost=26971.97..26971.98 rows=1 width=8) (actual time=821.368..823.052 rows=1.00 loops=1)"
" Buffers: shared hit=340234"
" -> Gather (cost=26971.75..26971.96 rows=2 width=8) (actual time=821.357..823.043 rows=3.00 loops=1)"
"   Workers Planned: 2"
"   Workers Launched: 2"
"     Buffers: shared hit=340234"
" -> Partial Aggregate (cost=25971.75..25971.76 rows=1 width=8) (actual time=816.313..816.316 rows=1.00 loops=3)"
"   Buffers: shared hit=340234"
"     -> Parallel Index Only Scan using idx_orders_order_date on orders (cost=0.43..24938.38 rows=413351 width=0) (actual time=0.036..441.121 rows=332900.67 loops=3)"
"       Index Cond: ((order_date >= '2023-01-01 00:00:00+00':timestamp with time zone) AND (order_date < '2024-01-01 00:00:00+00':timestamp with time zone))"
"       Heap Fetches: 0"
"       Index Searches: 1"
"       Buffers: shared hit=340234"
"Planning Time: 0.108 ms"
"Execution Time: 823.093 ms"
```

| Métrica          | Antes               | Después                  |
|------------------|---------------------|--------------------------|
| Tipo de Scan     | Parallel Seq Scan   | Parallel Index Only Scan |
| Heap Fetches     | millones implícitos | 0                        |
| Buffers (read)   | 41667               | 0 (solo hit en cache)    |
| Buffers (hit)    | bajo                | 340234                   |
| Tiempo ejecución | 1644 ms             | 823 ms                   |
| Sargabilidad     | No                  | Si                       |

### 3. Caso de optimización: Construcción y Optimización de la Consulta Compleja (JOIN + WHERE + GROUP BY)

Consulta: Queremos saber el Top 5 de ciudades que más dinero han gastado en productos de la categoría 'Electrónica' durante el último año.

```
-- Q3 Consulta que hace uso de JOIN + WHERE + GROUP BY
WITH electronics AS (
  SELECT product_id
  FROM product
  WHERE category = 'Electrónica'
),
orders_year AS (
  SELECT order_id, customer_id
  FROM orders
  WHERE order_date >= '2024-01-01'::timestampz
  AND order_date < '2025-01-01'::timestampz -- recomendable limitar rango
),
sales_per_order AS (
  SELECT oi.order_id, SUM(oi.quantity * oi.unit_price) AS revenue
  FROM order_item oi
  JOIN electronics e ON oi.product_id = e.product_id
  GROUP BY oi.order_id
)
SELECT c.city, SUM(s.revenue) AS total_revenue
FROM sales_per_order s
JOIN orders_year o ON s.order_id = o.order_id
JOIN customer c ON o.customer_id = c.customer_id
GROUP BY c.city
ORDER BY total_revenue DESC
LIMIT 5;
```

Al realizar esta consulta tan amplia y compleja, nuestro motor PostgreSQL toma varios métodos que tiene por defecto, incluidos el Sort, Hash Join, Seq Scan, GroupAggregate, etc, por lo que, sin una correcta optimización, este tipo de consultas serian impensables para un usuario como nosotros que queremos las respuestas en ms, aquí se tardó hasta 28762.852 ms, lo cual no es justificable.

```

"Limit (cost=658123.18..658123.19 rows=5 width=41) (actual time=28211.218..28211.342 rows=5.00 loops=1)"
" Buffers: shared hit=1715 read=224105, temp read=8520 written=8523"
" -> Sort (cost=658123.18..658123.20 rows=10 width=41) (actual time=27759.783..27759.901 rows=5.00 loops=1)"
"   Sort Key: (sum(s.revenue)) DESC"
"   Sort Method: quicksort Memory: 25kB"
"   Buffers: shared hit=1715 read=224105, temp read=8520 written=8523"
" -> HashAggregate (cost=658122.88..658123.01 rows=10 width=41) (actual time=27759.716..27759.839 rows=10.00 loops=1)"
"   Group Key: c.city"
"   Batches: 1 Memory Usage: 32kB"
"   Buffers: shared hit=1712 read=224105, temp read=8520 written=8523"
" -> HashJoin (cost=628770.41..656986.39 rows=227298 width=41) (actual time=25738.703..27577.258 rows=208863.00 loops=1)"
"   Hash Cond: (c.customer_id = orders.customer_id)"
"   Buffers: shared hit=1712 read=224105, temp read=8520 written=8523"
" -> Seq Scan on customer c (cost=0.00..22193.00 rows=1000000 width=17) (actual time=0.013..688.571 rows=1000000.00 loops=1)"
"   Buffers: shared hit=2 read=12191"
" -> Hash (cost=625929.19..625929.19 rows=227298 width=40) (actual time=25737.673..25737.784 rows=208863.00 loops=1)"
"   Buckets: 262144 Batches: 1 Memory Usage: 11839kB"
"   Buffers: shared hit=1710 read=211914, temp read=8520 written=8523"
" -> HashJoin (cost=427051.95..625929.19 rows=227298 width=40) (actual time=19287.138..25533.404 rows=208863.00 loops=1)"
"   Hash Cond: (s.order_id = orders.order_id)"
"   Buffers: shared hit=1710 read=211914, temp read=8520 written=8523"
" -> Subquery Scan on s (cost=331172.76..504601.05 rows=1123787 width=40) (actual time=17373.618..22184.944 rows=1043464.00 loops=1)"
"   Buffers: shared hit=1710 read=167514, temp read=4032 written=4035"
" -> GroupAggregate (cost=331172.76..493363.18 rows=1123787 width=40) (actual time=17373.613..20797.933 rows=1043464.00 loops=1)"
"     Group Key: oi.order_id"
"     Buffers: shared hit=1710 read=167514, temp read=4032 written=4035"
" -> Gather Merge (cost=331172.76..467601.82 rows=1171402 width=18) (actual time=17373.561..18793.646 rows=1136391.00 loops=1)"
"       Workers Planned: 2"
"       Workers Launched: 2"
"       Buffers: shared hit=1710 read=167514, temp read=4032 written=4035"
" -> Sort (cost=330172.73..331392.94 rows=488084 width=18) (actual time=17341.282..17601.491 rows=378797.00 loops=3)"
"         Sort Key: oi.order_id"
"         Sort Method: external merge Disk: 10784kB"
"         Buffers: shared hit=1710 read=167514, temp read=4032 written=4035"
"         Worker 0: Sort Method: external merge Disk: 11120kB"
"         Worker 1: Sort Method: external merge Disk: 10352kB"
" -> HashJoin (cost=2170.21..274046.68 rows=488084 width=18) (actual time=170.379..16810.625 rows=378797.00 loops=3)"
"   Hash Cond: (oi.product_id = product.product_id)"
"   Buffers: shared hit=1694 read=167514"
" -> Parallel Seq Scan on order_item oi (cost=0.00..250000.50 rows=8333350 width=26) (actual time=1.323..7954.184 rows=6666666.67 loops=3)"
"     Buffers: shared read=166667"
" -> Hash (cost=2097.00..2097.00 rows=5857 width=8) (actual time=168.972..168.975 rows=5688.00 loops=3)"
"   Buckets: 8192 Batches: 1 Memory Usage: 287kB"
"   Buffers: shared hit=1694 read=847"
" -> Seq Scan on product (cost=0.00..2097.00 rows=5857 width=8) (actual time=144.154..161.041 rows=5688.00 loops=3)"
"     Filter: (category = 'Electrónica':text)"
"     Rows Removed by Filter: 94312"
"     Buffers: shared hit=1694 read=847"
" -> Hash (cost=78298.88..78298.88 rows=1011305 width=16) (actual time=1908.517..1908.521 rows=999467.00 loops=1)"
"   Buckets: 1048576 Batches: 2 Memory Usage: 31639kB"
"   Buffers: shared read=44400, temp written=2194"
" -> Bitmap Heap Scan on orders (cost=21462.31..78298.88 rows=1011305 width=16) (actual time=189.168..1032.570 rows=999467.00 loops=1)"
"   Recheck Cond: ((order_date >= '2024-01-01 00:00:00+00':timestamp with time zone) AND (order_date < '2025-01-01 00:00:00+00':timestamp with time zone))"
"   Heap Blocks: exact=41667"
"   Buffers: shared read=44400"
" -> Bitmap Index Scan on idx_orders_order_date (cost=0.00..21209.48 rows=1011305 width=0) (actual time=179.235..179.236 rows=999467.00 loops=1)"
"   Index Cond: ((order_date >= '2024-01-01 00:00:00+00':timestamp with time zone) AND (order_date < '2025-01-01 00:00:00+00':timestamp with time zone))"
"   Index Searches: 1"
"   Buffers: shared read=2733"

"Planning:"
" Buffers: shared hit=237 read=16"
"Planning Time: 15.300 ms"
"JIT:"
" Functions: 67"
" Options: Inlining true, Optimization true, Expressions true, Deforming true"
" Timing: Generation 3.977 ms (Deform 1.347 ms), Inlining 322.187 ms, Optimization 303.267 ms, Emission 257.751 ms, Total 887.182 ms"
"Execution Time: 28762.892 ms"

```

Al observar lo costoso que es esta consulta sin optimización, proponemos el uso de los siguientes índices:

```
-- indices propuestos:
CREATE INDEX CONCURRENTLY idx_product_cat_electronica
  ON product(product_id)
  WHERE category = 'Electrónica';

CREATE INDEX CONCURRENTLY idx_orders_order_date_customer
  ON orders (order_date) INCLUDE (customer_id);

CREATE INDEX CONCURRENTLY idx_order_item_product_id
  ON order_item(product_id);

CREATE INDEX CONCURRENTLY idx_order_item_prod_includes
  ON order_item(product_id)
  INCLUDE (order_id, quantity, unit_price);
```

- idx\_product\_cat\_electronica Se creó como índice parcial para almacenar únicamente los productos cuya categoría es 'Electrónica'.
- idx\_orders\_order\_date\_customer Se creó para optimizar el filtro por rango de fechas (order\_date >= '2024-01-01') y evitar accesos adicionales a la tabla para obtener customer\_id.
- idx\_order\_item\_product\_id Se creó para acelerar el JOIN entre order\_item y product mediante la columna product\_id.
- idx\_order\_item\_prod\_includes Se creó para permitir que el cálculo SUM(quantity \* unit\_price) y el JOIN con orders puedan resolverse directamente desde el índice.

Tras crear un índice parcial en product para la categoría 'Electrónica' y un índice INCLUDE en order\_item (product\_id INCLUDE (order\_id, quantity, unit\_price)) y en orders(order\_date INCLUDE customer\_id), la consulta experimentó una mejora significativa:

```
"Limit (cost=337167.66..337167.67 rows=5 width=41) (actual time=16074.936..16093.469 rows=5.00 loops=1)"
" Buffers: shared hit=180160 read=67287, temp read=28508 written=45248"
" -> Sort (cost=337167.66..337167.68 rows=10 width=41) (actual time=16058.332..16076.859 rows=5.00 loops=1)"
"   Sort Key: (sum(s.revenue)) DESC"
"   Sort Method: quicksort Memory: 25kB"
"   Buffers: shared hit=180160 read=67287, temp read=28508 written=45248"
" -> Finalize GroupAggregate (cost=337164.39..337167.49 rows=10 width=41) (actual time=16058.240..16076.829 rows=10.00 loops=1)"
"   Group Key: c.city"
"   Buffers: shared hit=180160 read=67287, temp read=28508 written=45248"
" -> Gather Merge (cost=337164.39..337167.19 rows=24 width=41) (actual time=16058.201..16076.755 rows=30.00 loops=1)"
"   Workers Planned: 2"
"   Workers Launched: 2"
"   Buffers: shared hit=180160 read=67287, temp read=28508 written=45248"
" -> Sort (cost=336164.37..336164.39 rows=10 width=41) (actual time=16009.038..16009.062 rows=10.00 loops=3)"
"   Sort Key: c.city"
"   Sort Method: quicksort Memory: 25kB"
"   Buffers: shared hit=180160 read=67287, temp read=28508 written=45248"
"   Worker 0: Sort Method: quicksort Memory: 25kB"
"   Worker 1: Sort Method: quicksort Memory: 25kB"
" -> Partial HashAggregate (cost=336164.08..336164.20 rows=10 width=41) (actual time=16008.228..16008.253 rows=10.00 loops=3)"
"   Group Key: c.city"
"   Batches: 1 Memory Usage: 32kB"
"   Buffers: shared hit=180144 read=67287, temp read=28508 written=45248"
"   Worker 0: Batches: 1 Memory Usage: 32kB"
"   Worker 1: Batches: 1 Memory Usage: 32kB"
" -> Parallel Hash Join (cost=317373.75..335690.54 rows=94708 width=41) (actual time=14850.628..15908.021 rows=69621.00 loops=3)"
"   Hash Cond: (c.customer_id = orders.customer_id)"
"   Buffers: shared hit=180144 read=67287, temp read=28508 written=45248"
" -> Parallel Seq Scan on customer c (cost=0.00..16359.67 rows=416667 width=17) (actual time=0.356..448.713 rows=333333.33 loops=3)"
"   Buffers: shared read=12193"
" -> Parallel Hash (cost=316189.90..316189.90 rows=94708 width=40) (actual time=14849.521..14849.534 rows=69621.00 loops=3)"
"   Buckets: 262144 Batches: 1 Memory Usage: 11936kB"
```

```

"
" Buffers: shared hit=180144 read=55094, temp read=28508 written=45248"
"
" -> Hash Join (cost=254200.13..316189.90 rows=94708 width=40) (actual time=13367.923..14671.804 rows=69621.00 loops=3)"
"   Hash Cond: (orders.order_id = s.order_id)"
"   Buffers: shared hit=180144 read=55094, temp read=28508 written=45248"
"   -> Parallel Bitmap Heap Scan on orders (cost=21462.31..69449.96 rows=421377 width=16) (actual time=86.723..635.886 rows=333155.67 loops=3)"
"     Recheck Cond: ((order_date >= '2024-01-01 00:00:00+00':timestamp with time zone) AND (order_date < '2025-01-01 00:00:00+00':timestamp with time zone))"
"     Heap Blocks: exact=22587"
"     Buffers: shared read=44400"
"     Worker 0: Heap Blocks: exact=19077"
"     Worker 1: Heap Blocks: exact=3"
"     -> Bitmap Index Scan on idx_orders_order_date (cost=0.00..21209.48 rows=1011305 width=0) (actual time=86.953..86.954 rows=999467.00 loops=1)"
"       Index Cond: ((order_date >= '2024-01-01 00:00:00+00':timestamp with time zone) AND (order_date < '2025-01-01 00:00:00+00':timestamp with time zone))"
"       Index Searches: 1"
"       Buffers: shared read=2733"
"     -> Hash (cost=209910.48..209910.48 rows=1123787 width=40) (actual time=13276.352..13276.360 rows=1043464.00 loops=3)"
"       Buckets: 524288 Batches: 4 Memory Usage: 16294kB"
"       Buffers: shared hit=180084 read=10690, temp read=14877 written=41940"
"     -> Subquery Scan on s (cost=170897.93..209910.48 rows=1123787 width=40) (actual time=6324.755..11827.088 rows=1043464.00 loops=3)"
"       Buffers: shared hit=180084 read=10690, temp read=14877 written=31617"
"       -> HashAggregate (cost=170897.93..198672.61 rows=1123787 width=40) (actual time=6324.750..9467.561 rows=1043464.00 loops=3)"
"         Group Key: oi.order_id"
"         Planned Partitions: 16 Batches: 17 Memory Usage: 32913kB Disk Usage: 47592kB"
"         Buffers: shared hit=180084 read=10690, temp read=14877 written=31617"
"         Worker 0: Batches: 17 Memory Usage: 32913kB Disk Usage: 47592kB"
"         Worker 1: Batches: 17 Memory Usage: 32913kB Disk Usage: 47592kB"
"       -> Nested Loop (cost=0.84..80846.56 rows=1171400 width=18) (actual time=0.087..4083.977 rows=1136391.00 loops=3)"
"         Buffers: shared hit=180084 read=10690"
"         -> Index Only Scan using idx_product_cat_electronica on product (cost=0.28..159.29 rows=5857 width=8) (actual time=0.045..8.149 rows=5688.00
loops=3)"
"           Heap Fetches: 0"
"           Index Searches: 3"
"           Buffers: shared hit=39 read=17"
"         -> Index Only Scan using idx_order_item_prod_includes on order_item oi (cost=0.56..11.75 rows=203 width=26) (actual time=0.019..0.277 rows=199.79
loops=17064)"
"           Index Cond: (product_id = product.product_id)"
"           Heap Fetches: 0"
"           Index Searches: 17064"
"           Buffers: shared hit=180045 read=10673"
"
"Planning:"
" Buffers: shared hit=61 read=26 dirtied=1"
"Planning Time: 1.926 ms"
"JIT:"
" Functions: 124"
" Options: Inlining false, Optimization false, Expressions true, Deforming true"
" Timing: Generation 7.933 ms (Deform 1.571 ms), Inlining 0.000 ms, Optimization 2.873 ms, Emission 115.009 ms, Total 125.816 ms"
"Execution Time: 16104.480 ms"

```

- Tiempo de ejecución: 28.2 s → 16.1 s.
- Lecturas desde disco (buffers read): 224.105 → 67.287 (reducción ~70%).
- Se habilitaron Index Only Scans en product y order\_item (Heap Fetches = 0), lo que evita accesos al heap y mejora la latencia.
- El número de filas agregadas sigue siendo grande (≈1M grupos), por lo que la fase de agregación sigue siendo costosa y actualmente usa espacio temporal en disco.

Conclusión: la estrategia de índices mejoró drásticamente el acceso y redujo I/O de tablas.



| Métrica                      | Antes (Sin índices especializados) | Después (Con índices aplicados)    | Impacto                            |
|------------------------------|------------------------------------|------------------------------------|------------------------------------|
| Tiempo de ejecución          | 28.211 ms                          | 16.104 ms                          | más rápido                         |
| Buffers – shared read        | 224.105                            | 67.287                             | menos lecturas desde disco         |
| Buffers – shared hit         | 1.715                              | 180.160                            | Mucho más uso de caché             |
| Temp read                    | 8.520                              | 28.508                             | Aumentó (spills en agregaciones)   |
| Temp written                 | 8.523                              | 45.248                             | Aumentó (uso de disco temporal)    |
| Acceso a product             | Seq Scan                           | Index Only Scan                    | Heap Fetches = 0                   |
| Acceso a order_item          | Parallel Seq Scan                  | Index Only Scan (INCLUDE)          | Heap Fetches = 0                   |
| Acceso a orders por fecha    | Bitmap Heap Scan                   | Parallel Bitmap Heap Scan          | Índice por rango usado             |
| Heap Fetches                 | Implícitos (altos)                 | 0 en nodos críticos                | Eliminados en product y order_item |
| Tipo de agregación principal | HashAggregate                      | Finalize GroupAggregate (paralelo) | Más eficiente                      |
| Paralelismo                  | 2 workers                          | 2 workers                          | Se mantiene                        |

#### 4.Caso de optimización: Query con JOIN + Agregación

Consulta:

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT o.status, SUM(oi.quantity * oi.unit_price) AS revenue
FROM orders2 o
JOIN order_item2 oi ON oi.order_id = o.order_id
WHERE o.order_date >= DATE '2024-06-01'
      AND o.order_date < DATE '2024-07-01'
GROUP BY o.status
ORDER BY revenue DESC;
```

|    |                                                                                                                                       |
|----|---------------------------------------------------------------------------------------------------------------------------------------|
| 1  | Sort (cost=78178.80..78178.81 rows=5 width=39) (actual time=4637.636..4644.275 rows=5.00 loops=1)                                     |
| 2  | Sort Key: (sum(((oi.quantity)::numeric * oi.unit_price))) DESC                                                                        |
| 3  | Sort Method: quicksort Memory: 25kB                                                                                                   |
| 4  | Buffers: shared hit=31906 read=13895                                                                                                  |
| 5  | -> Finalize GroupAggregate (cost=78177.19..78178.74 rows=5 width=39) (actual time=4637.589..4644.249 rows=5.00 loops=1)               |
| 6  | Group Key: o.status                                                                                                                   |
| 7  | Buffers: shared hit=31903 read=13895                                                                                                  |
| 8  | -> Gather Merge (cost=78177.19..78178.59 rows=12 width=39) (actual time=4637.573..4644.222 rows=15.00 loops=1)                        |
| 9  | Workers Planned: 2                                                                                                                    |
| 10 | Workers Launched: 2                                                                                                                   |
| 11 | Buffers: shared hit=31903 read=13895                                                                                                  |
| 12 | -> Sort (cost=77177.17..77177.18 rows=5 width=39) (actual time=4616.351..4616.362 rows=5.00 loops=3)                                  |
| 13 | Sort Key: o.status                                                                                                                    |
| 14 | Sort Method: quicksort Memory: 25kB                                                                                                   |
| 15 | Buffers: shared hit=31903 read=13895                                                                                                  |
| 16 | Worker 0: Sort Method: quicksort Memory: 25kB                                                                                         |
| 17 | Worker 1: Sort Method: quicksort Memory: 25kB                                                                                         |
| 18 | -> Partial HashAggregate (cost=77177.05..77177.11 rows=5 width=39) (actual time=4616.311..4616.322 rows=5.00 loops=3)                 |
| 19 | Group Key: o.status                                                                                                                   |
| 20 | Batches: 1 Memory Usage: 32kB                                                                                                         |
| 21 | Buffers: shared hit=31887 read=13895                                                                                                  |
| 22 | Worker 0: Batches: 1 Memory Usage: 32kB                                                                                               |
| 23 | Worker 1: Batches: 1 Memory Usage: 32kB                                                                                               |
| 24 | -> Parallel Hash Join (cost=7510.69..75427.57 rows=174948 width=19) (actual time=53.833..4361.194 rows=136543.00 loops=3)             |
| 25 | Hash Cond: (oi.order_id = o.order_id)                                                                                                 |
| 26 | Buffers: shared hit=31887 read=13895                                                                                                  |
| 27 | -> Parallel Seq Scan on order_item2 oi (cost=0.00..62448.14 rows=2083314 width=20) (actual time=0.577..1883.743 rows=1666666.67 lo... |
| 28 | Buffers: shared hit=27720 read=13895                                                                                                  |
| 29 | -> Parallel Hash (cost=7292.00..7292.00 rows=17495 width=15) (actual time=52.518..52.520 rows=13683.33 loops=3)                       |
| 30 | Buckets: 65536 Batches: 1 Memory Usage: 2624kB                                                                                        |
| 31 | Buffers: shared hit=4167                                                                                                              |
| 32 | -> Parallel Seq Scan on orders2 o (cost=0.00..7292.00 rows=17495 width=15) (actual time=0.012..31.267 rows=13683.33 loops=3)          |
| 33 | Filter: ((order_date >= '2024-06-01'::date) AND (order_date < '2024-07-01'::date))                                                    |
| 34 | Rows Removed by Filter: 152983                                                                                                        |
| 35 | Buffers: shared hit=4167                                                                                                              |
| 36 | Planning:                                                                                                                             |
| 37 | Buffers: shared hit=59 read=10                                                                                                        |
| 38 | Planning Time: 0.331 ms                                                                                                               |
| 39 | Execution Time: 4644.337 ms                                                                                                           |

Se ejecuto un Parallel Seq Scan tanto sobre order\_item2 como sobre orders2, luego un Parallel Hash Join y una agregación masiva antes del ordenamiento final, implicando un procesamiento de millones de filas debido a la ausencia de particionamiento

Post particionamiento:

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT o.status, SUM(oi.quantity * oi.unit_price) AS revenue
FROM orders2p o
JOIN order_item2p oi ON oi.order_id = o.order_id
WHERE o.order_date >= DATE '2024-06-01'
AND o.order_date < DATE '2024-07-01'
AND oi.order_date >= DATE '2024-06-01'
AND oi.order_date < DATE '2024-07-01'
GROUP BY o.status
ORDER BY revenue DESC;
```

|    |                                                                                                                                            |
|----|--------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | Sort (cost=8917.65..8917.67 rows=5 width=40) (actual time=476.208..479.683 rows=5.00 loops=1)                                              |
| 2  | Sort Key: (sum(((oi.quantity)::numeric * oi.unit_price))) DESC                                                                             |
| 3  | Sort Method: quicksort Memory: 25kB                                                                                                        |
| 4  | Buffers: shared hit=3788                                                                                                                   |
| 5  | -> Finalize GroupAggregate (cost=8916.05..8917.60 rows=5 width=40) (actual time=476.175..479.670 rows=5.00 loops=1)                        |
| 6  | Group Key: o.status                                                                                                                        |
| 7  | Buffers: shared hit=3788                                                                                                                   |
| 8  | -> Gather Merge (cost=8916.05..8917.44 rows=12 width=40) (actual time=476.163..479.647 rows=15.00 loops=1)                                 |
| 9  | Workers Planned: 2                                                                                                                         |
| 10 | Workers Launched: 2                                                                                                                        |
| 11 | Buffers: shared hit=3788                                                                                                                   |
| 12 | -> Sort (cost=7916.02..7916.03 rows=5 width=40) (actual time=468.300..468.312 rows=5.00 loops=3)                                           |
| 13 | Sort Key: o.status                                                                                                                         |
| 14 | Sort Method: quicksort Memory: 25kB                                                                                                        |
| 15 | Buffers: shared hit=3788                                                                                                                   |
| 16 | Worker 0: Sort Method: quicksort Memory: 25kB                                                                                              |
| 17 | Worker 1: Sort Method: quicksort Memory: 25kB                                                                                              |
| 18 | -> Partial HashAggregate (cost=7915.90..7915.96 rows=5 width=40) (actual time=468.256..468.266 rows=5.00 loops=3)                          |
| 19 | Group Key: o.status                                                                                                                        |
| 20 | Batches: 1 Memory Usage: 32kB                                                                                                              |
| 21 | Buffers: shared hit=3772                                                                                                                   |
| 22 | Worker 0: Batches: 1 Memory Usage: 32kB                                                                                                    |
| 23 | Worker 1: Batches: 1 Memory Usage: 32kB                                                                                                    |
| 24 | -> Parallel Hash Join (cost=1007.04..7751.10 rows=16480 width=20) (actual time=47.018..446.041 rows=11292.33 loops=3)                      |
| 25 | Hash Cond: (oi.order_id = o.order_id)                                                                                                      |
| 26 | Buffers: shared hit=3772                                                                                                                   |
| 27 | -> Parallel Seq Scan on order_item2p_2024_06 oi (cost=0.00..6003.49 rows=171633 width=20) (actual time=0.011..182.196 rows=137306.33 lo... |
| 28 | Filter: ((order_date >= '2024-06-01'::date) AND (order_date < '2024-07-01'::date))                                                         |
| 29 | Buffers: shared hit=3429                                                                                                                   |
| 30 | -> Parallel Hash (cost=705.21..705.21 rows=24147 width=16) (actual time=45.871..45.873 rows=13683.33 loops=3)                              |
| 31 | Buckets: 65536 Batches: 1 Memory Usage: 2592kB                                                                                             |
| 32 | Buffers: shared hit=343                                                                                                                    |
| 33 | -> Parallel Seq Scan on orders2p_2024_06 o (cost=0.00..705.21 rows=24147 width=16) (actual time=0.014..23.842 rows=13683.33 loops=3)       |
| 34 | Filter: ((order_date >= '2024-06-01'::date) AND (order_date < '2024-07-01'::date))                                                         |
| 35 | Buffers: shared hit=343                                                                                                                    |
| 36 | Planning:                                                                                                                                  |
| 37 | Buffers: shared hit=62                                                                                                                     |
| 38 | Planning Time: 0.582 ms                                                                                                                    |
| 39 | Execution Time: 479.752 ms                                                                                                                 |

| Métrica               | Antes (Sin particionamiento)      | Después (Con particionamiento)      | Impacto                        |
|-----------------------|-----------------------------------|-------------------------------------|--------------------------------|
| Execution Time        | 4644.337 ms                       | 4636.695 ms                         | Mejora marginal                |
| Buffers – shared read | 13,895                            | 18,018                              | Más lecturas físicas           |
| Buffers – shared hit  | 31,906                            | 27,819                              | Leve reducción en uso de caché |
| Temp usage            | No significativo                  | No significativo                    | Sin impacto relevante          |
| Acceso a order_item   | Parallel Seq Scan                 | Parallel Seq Scan (partición junio) | Se escanea solo la partición   |
| Acceso a orders       | Parallel Seq Scan                 | Parallel Bitmap Heap Scan           | Uso de índice por rango        |
| Filtro por fecha      | Aplicado sobre tabla completa     | Pruning de partición (junio)        | Reducción lógica del dataset   |
| Tipo de Join          | Parallel Hash Join                | Parallel Hash Join                  | Se mantiene                    |
| Tipo de agregación    | Partial + Finalize GroupAggregate | Partial + Finalize GroupAggregate   | Se mantiene                    |
| Paralelismo           | 2 workers                         | 2 workers                           | Se mantiene                    |

## 5. Caso de optimización: Top productos

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT oi.product_id, SUM(oi.quantity) AS units
FROM order_item2 oi
WHERE oi.order_date >= DATE '2024-06-01'
      AND oi.order_date < DATE '2024-07-01'
GROUP BY oi.product_id
ORDER BY units DESC
LIMIT 10;
```

|    |                                                                                                                                    |
|----|------------------------------------------------------------------------------------------------------------------------------------|
| 1  | Limit (cost=89290.30..89290.33 rows=10 width=16) (actual time=1010.468..1010.589 rows=10.00 loops=1)                               |
| 2  | Buffers: shared hit=27732 read=13883 written=1                                                                                     |
| 3  | -> Sort (cost=89290.30..89414.53 rows=49691 width=16) (actual time=1010.464..1010.575 rows=10.00 loops=1)                          |
| 4  | Sort Key: (sum(quantity)) DESC                                                                                                     |
| 5  | Sort Method: top-N heapsort Memory: 25kB                                                                                           |
| 6  | Buffers: shared hit=27732 read=13883 written=1                                                                                     |
| 7  | -> Finalize HashAggregate (cost=87719.59..88216.50 rows=49691 width=16) (actual time=935.395..977.320 rows=49988.00 loops=1)       |
| 8  | Group Key: product_id                                                                                                              |
| 9  | Batches: 1 Memory Usage: 3097kB                                                                                                    |
| 10 | Buffers: shared hit=27732 read=13883 written=1                                                                                     |
| 11 | -> Gather (cost=74700.59..87123.30 rows=119258 width=16) (actual time=625.099..797.225 rows=139500.00 loops=1)                     |
| 12 | Workers Planned: 2                                                                                                                 |
| 13 | Workers Launched: 2                                                                                                                |
| 14 | Buffers: shared hit=27732 read=13883 written=1                                                                                     |
| 15 | -> Partial HashAggregate (cost=73700.59..74197.50 rows=49691 width=16) (actual time=602.335..651.355 rows=46500.00 loops=3)        |
| 16 | Group Key: product_id                                                                                                              |
| 17 | Batches: 1 Memory Usage: 3097kB                                                                                                    |
| 18 | Buffers: shared hit=27732 read=13883 written=1                                                                                     |
| 19 | Worker 0: Batches: 1 Memory Usage: 3097kB                                                                                          |
| 20 | Worker 1: Batches: 1 Memory Usage: 3097kB                                                                                          |
| 21 | -> Parallel Seq Scan on order_item2 oi (cost=0.00..72864.71 rows=167176 width=12) (actual time=0.131..387.686 rows=137306.33 lo... |
| 22 | Filter: ((order_date >= '2024-06-01'::date) AND (order_date < '2024-07-01'::date))                                                 |
| 23 | Rows Removed by Filter: 1529360                                                                                                    |
| 24 | Buffers: shared hit=27732 read=13883 written=1                                                                                     |
| 25 | Planning:                                                                                                                          |
| 26 | Buffers: shared hit=9 dirtied=1                                                                                                    |
| 27 | Planning Time: 0.131 ms                                                                                                            |
| 28 | Execution Time: 1011.351 ms                                                                                                        |

Se ejecuto un Parallel Seq Scan sobre la tabla order\_item2, aplicando el filtro por rango de fechas y realizando posteriormente una agregación masiva seguida de un ordenamiento para obtener los 10 productos con mayor cantidad, esto implica un procesamiento completo del volumen.

Post particionamiento:

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT oi.product_id, SUM(oi.quantity) AS units
FROM order_item2p oi
WHERE oi.order_date >= DATE '2024-06-01'
      AND oi.order_date < DATE '2024-07-01'
GROUP BY oi.product_id
ORDER BY units DESC
LIMIT 10;
```

|    |                                                                                                                                    |
|----|------------------------------------------------------------------------------------------------------------------------------------|
| 1  | Limit (cost=13262.37..13262.40 rows=10 width=16) (actual time=775.201..775.221 rows=10.00 loops=1)                                 |
| 2  | Buffers: shared hit=3429                                                                                                           |
| 3  | -> Sort (cost=13262.37..13388.52 rows=50459 width=16) (actual time=775.199..775.208 rows=10.00 loops=1)                            |
| 4  | Sort Key: (sum(oi.quantity)) DESC                                                                                                  |
| 5  | Sort Method: top-N heapsort Memory: 25kB                                                                                           |
| 6  | Buffers: shared hit=3429                                                                                                           |
| 7  | -> HashAggregate (cost=11667.38..12171.97 rows=50459 width=16) (actual time=697.580..739.517 rows=49988.00 loops=1)                |
| 8  | Group Key: oi.product_id                                                                                                           |
| 9  | Batches: 1 Memory Usage: 3097kB                                                                                                    |
| 10 | Buffers: shared hit=3429                                                                                                           |
| 11 | -> Seq Scan on order_item2p_2024_06 oi (cost=0.00..9607.78 rows=411919 width=12) (actual time=0.011..305.219 rows=411919.00 loo... |
| 12 | Filter: ((order_date >= '2024-06-01'::date) AND (order_date < '2024-07-01'::date))                                                 |
| 13 | Buffers: shared hit=3429                                                                                                           |
| 14 | Planning:                                                                                                                          |
| 15 | Buffers: shared hit=17 read=1 dirtied=2                                                                                            |
| 16 | Planning Time: 0.230 ms                                                                                                            |
| 17 | Execution Time: 775.351 ms                                                                                                         |

| Métrica                | Antes (Sin particionamiento)       | Después (Con particionamiento)   | Impacto                           |
|------------------------|------------------------------------|----------------------------------|-----------------------------------|
| Execution Time         | 1011.351 ms (~1.01 s)              | 775.351 ms (~0.78 s)             | ~23% más rápido                   |
| Buffers – shared read  | 13,883                             | 0                                | Eliminadas lecturas físicas       |
| Buffers – shared hit   | 27,732                             | 3,429                            | Mucho menor uso de caché          |
| Rows removed by filter | 152,930                            | No significativo                 | Se evita escaneo innecesario      |
| Acceso a order_item    | Parallel Seq Scan (tabla completa) | Seq Scan sobre partición 2024_06 | Partition pruning aplicado        |
| Filtro por fecha       | Aplicado sobre toda la tabla       | Aplicado solo en partición junio | Menor volumen procesado           |
| Tipo de agregación     | Partial + Finalize HashAggregate   | HashAggregate directo            | Plan más simple                   |
| Sort                   | Top-N heapsort                     | Top-N heapsort                   | Se mantiene                       |
| Paralelismo            | 2 workers                          | No paralelo explícito            | Ya no necesario por menor volumen |

## Caso Empresarial: Optimización de Reportes en Mercado Libre

Una plataforma de comercio electrónico como Mercado Libre enfrentó problemas de rendimiento debido al crecimiento masivo de sus tablas de órdenes y detalle de productos. Los reportes por rango de fechas y consultas de productos más vendidos comenzaron a tardar varios segundos, especialmente cuando múltiples usuarios los ejecutaban simultáneamente.

El problema técnico se debía a escaneos completos (Seq Scan) sobre tablas grandes, JOIN masivos antes de aplicar agregaciones y falta de índices adecuados. Además, algunas consultas utilizaban funciones en el WHERE, impidiendo el uso eficiente de índices.

La solución implementada incluyó:

Creación de índices compuestos alineados con los filtros por fecha y estado.

Reescritura de consultas para agregar antes del JOIN y reducir volumen intermedio.

Particionamiento por rango de fechas para activar partition pruning.

Evaluación del rendimiento bajo concurrencia.

Esta situación es muy similar a la trabajada en nuestro proyecto, donde inicialmente se presentaron Parallel Seq Scan, Hash Join y altos tiempos de ejecución sobre millones de registros. Al aplicar índices, reescritura y particionamiento, se logró una mejora significativa en rendimiento y escalabilidad.

## Reevaluación de Consultas Optimizadas sobre Entorno Big Data

En esta sección se presentan nuevamente las consultas desarrolladas y optimizadas en el Informe 1.

A diferencia del escenario inicial, las pruebas actuales se realizan sobre un volumen de datos significativamente mayor (Big Data), con el objetivo de validar el comportamiento, escalabilidad y efectividad de las técnicas de optimización previamente justificadas.

Dado que el análisis conceptual, el fundamento teórico y la explicación detallada de cada técnica fueron desarrollados en el Informe 1, en esta sección se presentan únicamente los resultados comparativos obtenidos mediante mediciones antes y después de la optimización.

### Q1: Ventas por ciudad en un año

```
-- Q1: Ventas por ciudad en un año
EXPLAIN (ANALYZE, BUFFERS)
SELECT c.city, SUM(o.total_amount) AS total_sales
FROM customer c
JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_date >= TIMESTAMPTZ '2023-01-01'
      AND o.order_date < TIMESTAMPTZ '2024-01-01'
GROUP BY c.city
ORDER BY total_sales DESC;

-- Optimizacion:

-- Creamos un índice compuesto que incluye la columna del filtro (order_date),
-- la columna del JOIN (customer_id) y agregamos el monto (total_amount) como payload.
CREATE INDEX IF NOT EXISTS idx_orders_order_date_customer
ON orders (order_date, customer_id) INCLUDE (total_amount);
ANALYZE orders;
```

| Métrica                    | Antes (Sin índice) | Después (Index Only Scan) | Mejora             |
|----------------------------|--------------------|---------------------------|--------------------|
| Execution Time             | 3135.592 ms        | 2980.412 ms               | Disminuyo 4.9%     |
| Costo estimado (cost)      | 98627              | 59382                     | Disminuyo 39.8%    |
| Tipo de acceso a orders    | Parallel Seq Scan  | Parallel Index Only Scan  | Mejora estructural |
| Heap Fetches               | —                  | 0                         | Eliminados         |
| Buffers read               | 22532              | 7117                      | Disminuyo 68%      |
| Buffers hit                | 31344              | 347467                    | Mayor uso de caché |
| Rows procesadas por worker | 332900             | 332900                    | Igual.             |

Se eliminó el Parallel Seq Scan sobre la tabla orders y fue reemplazado por un Parallel Index Only Scan, lo que redujo significativamente las lecturas físicas en disco (buffers read ↓ 68%).

Aunque la reducción en tiempo total fue moderada (4.9%), la mejora estructural es importante, ya que en escenarios de mayor volumen o menor caché disponible el impacto sería considerablemente mayor.

Q2: Top productos vendidos:

```
-- Q2: Top productos vendidos sin optimizacion
EXPLAIN (ANALYZE, BUFFERS)
SELECT p.name, SUM(oi.quantity) AS total_sold
FROM order_item oi
JOIN product p ON oi.product_id = p.product_id
GROUP BY p.name
ORDER BY total_sold DESC
LIMIT 10;

-- Optimizacion:
-- Optimizacion de consulta Q2 sin indices, solo rescribiendo la consulta
EXPLAIN (ANALYZE, BUFFERS)
SELECT p.name, s.total_sold
FROM (
  SELECT product_id, SUM(quantity) AS total_sold
  FROM order_item
  GROUP BY product_id
) s
JOIN product p USING (product_id)
ORDER BY s.total_sold DESC
LIMIT 10;
```

| Métrica                       | Antes (JOIN + GROUP BY directo) | Después (PreAgregación) | Mejora             |
|-------------------------------|---------------------------------|-------------------------|--------------------|
| Execution Time                | 40,884.923 ms                   | 21,273.286 ms           | Disminuyo 47.9%    |
| Costo estimado (cost)         | 347000                          | 325910                  | Disminuyo 6%       |
| Filas procesadas en Hash Join | ~20M (6.6M x 3 workers)         | ~100k                   | Drástica reducción |
| Tipo de acceso a order_item   | Parallel Seq Scan               | Parallel Seq Scan       | Igual              |
| Buffers read                  | 167,514                         | 166,103                 | Leve mejora        |
| Rows finales agregadas        | 100,000                         | 100,000                 | Igual              |

La reescritura de la consulta aplicando pre-agregación sobre order\_item antes del JOIN redujo drásticamente el volumen de datos que participa en el Hash Join (de millones de filas a solo 100k agregadas).

Aunque el acceso a disco sigue siendo mediante Parallel Seq Scan, la reducción en cardinalidad intermedia permitió disminuir el tiempo total de ejecución en casi 48%, demostrando el impacto estructural de la optimización lógica incluso sin uso de índices.

Q3: Ultimas órdenes de un cliente

```
-- Q3: Dashboard: últimas órdenes de un cliente
EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM orders
WHERE customer_id = 12345
ORDER BY order_date DESC
LIMIT 20;

-- Optimizacion:

-- Q3: Índice compuesto: primero por la columna de igualdad (customer_id)
-- y luego por la columna de ordenamiento en la dirección solicitada (DESC).
CREATE INDEX IF NOT EXISTS idx_orders_customer_date_desc
ON orders(customer_id, order_date DESC);
ANALYZE orders;
```



| Métrica                | Antes (Sin índice)   | Después (Índice compuesto DESC) | Mejora                    |
|------------------------|----------------------|---------------------------------|---------------------------|
| Execution Time         | 270.177 ms           | 0.071 ms                        | Disminuyo <b>99.97%</b> 🚀 |
| Costo estimado (cost)  | 68708                | 28.54                           | Disminuyo 99.95%          |
| Tipo de Scan           | Parallel Seq Scan    | Index Scan                      | Eliminado Seq Scan        |
| Ordenamiento (Sort)    | Sí (quicksort)       | No necesario                    | Eliminado                 |
| Rows Removed by Filter | 1,666,666 por worker | 0                               | Eliminado                 |
| Buffers read           | 22,277               | 5                               | Disminuyo 99.97%          |
| Buffers hit            | 19,466               | 4                               | Drástica reducción        |

La creación del índice compuesto (customer\_id, order\_date DESC) permitió:

- Eliminar completamente el Parallel Seq Scan
- Evitar el Sort explícito
- Resolver la consulta directamente desde el índice

El tiempo de ejecución pasó de 270 ms a 0.071 ms, lo que representa una mejora superior al 99.9%, demostrando el impacto crítico de diseñar índices alineados con filtros y ordenamientos en consultas tipo dashboard.

Q4:

```
--Q4: Degradación típica: LIKE con comodín inicial (no sargable)
EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM product
WHERE name ILIKE '%42%'
LIMIT 50;

-- Optimizacion:

-- Habilitamos la extensión de trigramas
CREATE EXTENSION IF NOT EXISTS pg_trgm;
-- Creamos un índice GIN (Generalized Inverted Index) basado en trigramas
CREATE INDEX IF NOT EXISTS idx_product_name_trgm
ON product USING gin (name gin_trgm_ops);

CREATE INDEX IF NOT EXISTS idx_product_name_trgm_lower
ON product USING gin (lower(name) gin_trgm_ops);

EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM product
WHERE lower(name) LIKE '%42%'
LIMIT 50;
```

| Métrica                | Antes (ILIKE '%42%') | Después (GIN Trigram + lower) | Mejora       |
|------------------------|----------------------|-------------------------------|--------------|
| Execution Time         | 1.224 ms             | 0.835 ms                      | 31.8%        |
| Tipo de Scan           | Seq Scan             | Seq Scan                      | No cambió    |
| Costo estimado (cost)  | 2097                 | 2347                          | aumento      |
| Buffers hit            | 22                   | 22                            | igual        |
| Rows Removed by Filter | 2375                 | 2375                          | igual        |
| Uso del índice trigram | No                   | No                            | No utilizado |

Aunque se creó un índice GIN basado en trigramas, el plan de ejecución sigue utilizando Seq Scan, lo que indica que PostgreSQL decidió no usar el índice.

Esto ocurre porque:

- La tabla product no es lo suficientemente grande para que el índice sea más eficiente.
- El LIMIT 50 permite que el motor encuentre coincidencias rápidamente sin necesidad de indexación.
- El costo estimado del índice no compensa el uso frente al escaneo secuencial.

En este escenario Big Data específico, el índice no aporta mejora estructural significativa.

Por lo tanto no todo índice creado garantiza que el planner lo utilice, su uso depende del costo estimado y del volumen real de datos.

Q5:

```
-- Q5: Anti-pattern: función sobre columna en WHERE (rompe uso directo de índice)
EXPLAIN (ANALYZE, BUFFERS)
SELECT count(*)
FROM orders
WHERE date_trunc('day', order_date) = TIMESTAMPTZ '2023-11-15';

-- Optimizacion:

-- Q5: reescribir consulta para usar rango
-- Crea un índice B-Tree normal:
CREATE INDEX idx_orders_order_date ON orders (order_date);
-- Reescribir consulta, en lugar de usar = con una función,
-- usar operadores de rango (>= y <):
EXPLAIN (ANALYZE, BUFFERS)
SELECT count(*)
FROM orders
WHERE order_date >= '2023-11-15 00:00:00'::TIMESTAMPTZ
AND order_date < '2023-11-16 00:00:00'::TIMESTAMPTZ;
```

| Métrica                | Antes<br>(date_trunc) | Después (Rango + Índice B-Tree) | Mejora                   |
|------------------------|-----------------------|---------------------------------|--------------------------|
| Execution Time         | 1611.479 ms           | 4.334 ms                        | Disminuyo 99.73%         |
| Costo estimado (cost)  | 73943                 | 91                              | Disminuyo ~99.87%        |
| Tipo de Scan           | Parallel Seq Scan     | Index Only Scan                 | Eliminado Seq Scan       |
| Heap Fetches           | —                     | 0                               | Eliminados               |
| Rows Removed by Filter | 1,665,759             | 0                               | Eliminado                |
| Buffers read           | 41,667                | 10                              | Disminuyo 99.97%         |
| Buffers hit            | —                     | 977                             | Uso eficiente del índice |

El uso de date\_trunc() hacía que la consulta fuera no sargable, impidiendo el uso de índices y forzando un Parallel Seq Scan sobre toda la tabla.

Al reescribir la consulta utilizando un rango de fechas ( $\geq$  y  $<$ ) y crear un índice B-Tree sobre order\_date, PostgreSQL pudo ejecutar un Index Only Scan, eliminando completamente el escaneo secuencial.

El tiempo de ejecución pasó de 1.6 segundos a 4 ms, representando una mejora superior al 99%, demostrando el impacto crítico de la sargabilidad en entornos Big Data.

Q6:

```
-- Q6: Join + filtro por status (sin índices)
EXPLAIN (ANALYZE, BUFFERS)
SELECT o.status, count(*) AS n
FROM orders o
JOIN payment p ON p.order_id = o.order_id
WHERE p.payment_status = 'APPROVED'
GROUP BY o.status
ORDER BY n DESC;

-- Optimizacion:

-- Q6:
-- Optimizar la tabla payment (El filtro)
CREATE INDEX idx_payment_status_order_id ON payment (payment_status, order_id);

-- Índice de Cobertura en Orders
CREATE INDEX idx_orders_order_id_status_include ON orders (order_id, status);
ANALYZE orders;
```

| Métrica                          | Antes (Sin índices) | Después (Índices en payment y orders) | Mejora             |
|----------------------------------|---------------------|---------------------------------------|--------------------|
| Execution Time                   | 7750.774 ms         | 6825.655 ms                           | Disminuyo 11.9%    |
| Costo estimado (cost)            | 137386              | 110038                                | Disminuyo 19.9%    |
| Tipo de acceso a payment         | Parallel Seq Scan   | Parallel Index Only Scan              | Mejora estructural |
| Tipo de acceso a orders          | Parallel Seq Scan   | Parallel Seq Scan                     | Igual              |
| Rows Removed by Filter (payment) | 1,000,223           | 0 (filtrado por índice)               | Eliminado          |
| Buffers read                     | 76,911              | 44,081                                | Disminuyo 42.7%    |
| Heap Fetches (payment)           | —                   | 0                                     | Eliminados         |

La creación del índice compuesto en payment (payment\_status, order\_id) permitió:

- Eliminar el Seq Scan sobre payment
- Convertirlo en un Parallel Index Only Scan
- Reducir significativamente las lecturas en disco (buffers ↓ 42%)

Aunque la tabla orders sigue usando Parallel Seq Scan, el filtro principal ahora se resuelve eficientemente desde el índice, reduciendo el costo total y el tiempo de ejecución en casi 12%.

## Diferencias con RDS

Al crear una base de datos en RDS lastimosamente contamos con una restricción a la hora de crear la instancia, y es que la licencia nos limitó a usar un modelo db.t3.micro, el cual es lento en comparación con t2.large que está en ec2, es por esto que los queries que mostraremos a continuación serán un poco más lentos en comparación que los de ec2, cabe resaltar que las consultas son exactamente las mismas.

Query 1:

```
Proyecto1=> EXPLAIN (ANALYZE, BUFFERS)
Proyecto1-> SELECT name, email
Proyecto1-> FROM customer
Proyecto1-> WHERE email LIKE '%customer123%';
```

Antes de optimizar la consulta:

```
~ $ cat Q1_before.txt

QUERY PLAN
-----
Gather  (cost=1000.00..18411.33 rows=100 width=41) (actual time=10.148..666.332 rows=1111 loops=1)
  Workers Planned: 2
  Workers Launched: 2
  Buffers: shared hit=192 read=12001
  I/O Timings: shared read=901.509
  -> Parallel Seq Scan on customer  (cost=0.00..17401.33 rows=42 width=41) (actual time=174.274..366.960 rows=370 loops=3)
    Filter: (email ~ '%customer123% '::text)
    Rows Removed by Filter: 332963
    Buffers: shared hit=192 read=12001
    I/O Timings: shared read=901.509
  Planning Time: 3.609 ms
  Execution Time: 669.531 ms
(12 rows)
```

despues de optimizar la consulta:

```
QUERY PLAN
-----
Bitmap Heap Scan on customer  (cost=210.67..584.75 rows=100 width=41) (actual time=35.221..36.291 rows=1111 loops=1)
  Recheck Cond: (email ~ '%customer123% '::text)
  Rows Removed by Index Recheck: 20
  Heap Blocks: exact=29
  Buffers: shared hit=1655
  -> Bitmap Index Scan on idx_cust_email_trgm  (cost=0.00..210.64 rows=100 width=0) (actual time=35.194..35.195 rows=1131 loops=1)
    Index Cond: (email ~ '%customer123% '::text)
    Buffers: shared hit=1626
  Planning:
    Buffers: shared hit=1
  Planning Time: 0.091 ms
  Execution Time: 36.367 ms
(12 rows)
```

EC2 VS RDS:

Antes de optimizar la consulta Q1:

| Métrica        | RDS               | EC2               |
|----------------|-------------------|-------------------|
| Tipo de Scan   | Parallel Seq Scan | Parallel Seq Scan |
| Buffers        | 12001             | 12193             |
| Execution Time | 669 ms            | 81 ms             |

Después de optimizar la consulta:

| Métrica        | RDS              | EC2              |
|----------------|------------------|------------------|
| Tipo de Scan   | Bitmap Heap Scan | Bitmap Heap Scan |
| Buffers        | 1655             | 1667             |
| Heap Blocks    | 29               | 29               |
| Execution Time | 36 ms            | 28 ms            |

En ambos entornos, antes de la optimización, PostgreSQL ejecutó un Parallel Sequential Scan, leyendo la totalidad de la tabla ( $\approx 1\text{M}$  registros), con más de 12.000 buffers accedidos.

Sin embargo, el tiempo de ejecución en RDS (669 ms) fue considerablemente mayor

que en EC2 (81 ms), lo que sugiere diferencias en latencia de disco o configuración de infraestructura.

Tras crear el índice GIN basado en pg\_trgm, ambos entornos cambiaron el plan a Bitmap Heap Scan + Bitmap Index Scan, accediendo únicamente a 29 heap blocks.

Conclusión: La diferencia de tiempos observada entre RDS y EC2 puede explicarse principalmente por la capacidad de cómputo disponible en cada entorno. En RDS, la instancia utilizada fue db.t3.micro, la cual posee recursos limitados de CPU y memoria debido a restricciones de licencia del laboratorio. En contraste, en EC2 se empleó una instancia t2.large, con mayor capacidad de procesamiento.

Esto demuestra que, aunque la optimización mediante índices mejora significativamente el rendimiento en ambos entornos, la infraestructura subyacente (CPU, memoria y capacidad de I/O) influye directamente en los tiempos de ejecución absolutos.

Veamos otro caso, probemos con el query número 2:

Q2:

```
Proyecto1=> \o Q2_before.txt
Proyecto1=> EXPLAIN (ANALYZE, BUFFERS)
Proyecto1-> SELECT COUNT(*)
Proyecto1-> FROM orders
Proyecto1-> WHERE EXTRACT(YEAR FROM order_date) = 2023;
```

Antes de optimizar la consulta:

```
QUERY PLAN
-----
Finalize Aggregate  (cost=73943.16..73943.17 rows=1 width=8) (actual time=3021.271..3026.053 rows=1 loops=1)
  Buffers: shared hit=7437 read=34230
  I/O Timings: shared read=2684.686
    -> Gather  (cost=73942.94..73943.15 rows=2 width=8) (actual time=3020.237..3026.041 rows=3 loops=1)
        Workers Planned: 2
        Workers Launched: 2
        Buffers: shared hit=7437 read=34230
        I/O Timings: shared read=2684.686
          -> Partial Aggregate  (cost=72942.94..72942.95 rows=1 width=8) (actual time=3006.914..3006.915 rows=1 loops=3)
              Buffers: shared hit=7437 read=34230
              I/O Timings: shared read=2684.686
                -> Parallel Seq Scan on orders  (cost=0.00..72916.90 rows=10417 width=0) (actual time=1.559..2984.977 rows=333339 loops=3)
                    Filter: (EXTRACT(year FROM order_date) = '2023'::numeric)
                    Rows Removed by Filter: 1333327
                    Buffers: shared hit=7437 read=34230
                    I/O Timings: shared read=2684.686
Planning:
  Buffers: shared hit=55 read=5
  I/O Timings: shared read=7.062
Planning Time: 9.848 ms
Execution Time: 3026.172 ms
(21 rows)
```

Después de optimizar la consulta:

```
----- QUERY PLAN -----
Finalize Aggregate (cost=27295.36..27295.37 rows=1 width=8) (actual time=559.230..562.409 rows=1 loops=1)
  Buffers: shared hit=342144
  -> Gather (cost=27295.15..27295.36 rows=2 width=8) (actual time=558.413..562.400 rows=3 loops=1)
    Workers Planned: 2
    Workers Launched: 2
    Buffers: shared hit=342144
    -> Partial Aggregate (cost=26295.15..26295.16 rows=1 width=8) (actual time=539.642..539.643 rows=1 loops=3)
      Buffers: shared hit=342144
      -> Parallel Index Only Scan using idx_orders_order_date on orders (cost=0.43..25248.94 rows=418485 width=0) (actual time=3.156..518.354 rows=333339 loops=3)
        Index Cond: ((order_date >= '2023-01-01 00:00:00+00'::timestamp with time zone) AND (order_date < '2024-01-01 00:00:00+00'::timestamp with time zone))
        Heap Fetches: 0
        Buffers: shared hit=342144
Planning:
  Buffers: shared hit=50 read=3 dirtied=1
  I/O Timings: shared read=5.019
Planning Time: 24.678 ms
Execution Time: 567.931 ms
(17 rows)
```

EC2 VS RDS:

Antes de optimizar la consulta Q2:

| Métrica        | RDS                   | EC2               |
|----------------|-----------------------|-------------------|
| Tipo de Scan   | Parallel Seq Scan     | Parallel Seq Scan |
| Workers        | 2                     | 2                 |
| Buffers (read) | 34230                 | 41667             |
| Buffers (hit)  | 7437                  | Bajo              |
| Heap Fetches   | Implícitos (millones) | Implícitos        |
| I/O Timings    | 2684 ms               | No reportado      |
| Execution Time | <b>3026 ms</b>        | <b>1644 ms</b>    |
| Sargabilidad   | No                    | No                |

Después de optimizar la consulta:

| Métrica        | RDS                      | EC2                      |
|----------------|--------------------------|--------------------------|
| Tipo de Scan   | Parallel Index Only Scan | Parallel Index Only Scan |
| Workers        | 2                        | 2                        |
| Buffers (read) | 0                        | 0                        |
| Buffers (hit)  | 342144                   | 340234                   |
| Heap Fetches   | 0                        | 0                        |
| Acceso a Heap  | No                       | No                       |
| Execution Time | <b>567 ms</b>            | <b>823 ms</b>            |
| Sargabilidad   | Sí                       | Sí                       |

Después de optimizar la consulta, ambas instancias utilizaron el mismo plan de ejecución (Parallel Index Only Scan) y eliminaron completamente el acceso a disco. En este escenario, la consulta dejó de estar limitada por I/O y pasó a depender principalmente del estado de memoria, caché y gestión interna del motor.

Aunque la instancia EC2 (t2.large) dispone de mayor capacidad teórica de CPU, al ser una instancia burstable puede estar sujeta a limitaciones por créditos de CPU. Además, la configuración administrada de RDS puede ofrecer mejor eficiencia en el uso de memoria compartida y menor overhead del sistema operativo.

Por ello, la diferencia observada no contradice la capacidad de hardware, sino que refleja condiciones de ejecución específicas, estado de caché y características del tipo de instancia.